# Mitigating Model Uncertainty in Self-Adaptive Systems via Stacking-Based Ensemble

ResearchOS Autonomous Research Pipeline

April 19, 2026

**Abstract**

Model-selection uncertainty in self-adaptive systems can degrade prediction quality in the Analyze stage of a MAPE-K loop and thereby harm downstream adaptation decisions. This paper investigates whether replacing Bayesian Model Averaging (BMA) with a stacking-based ensemble meta-learner can improve prediction quality and adaptation quality in the robotics case study used by *Taming Model Uncertainty in Self-adaptive Systems Using Bayesian Model Averaging*. Our intended design keeps the candidate generalized linear model space, data source, preprocessing, split strategy, and planning logic fixed, and changes only the Analyze-stage combiner. The executed pipeline artifacts, however, show an important tension: the experiment plan and milestone reports repeatedly state that the study is blocked without the exact reference traces, while the final run artifacts and user directives indicate that a file-based rescue-robot dataset was available and that a comparison was executed. On the available run, BMA achieved a Brier score of 0.1716, log loss of 0.5122, and accuracy of 0.7338, while stacking achieved a Brier score of 0.1727, log loss of 0.5194, and accuracy of 0.7359 over 924 rows with split seed 42. Thus, stacking did not improve probabilistic accuracy on this observed run, though it slightly increased classification accuracy. Because adaptation-level metrics such as requirement-violating adaptation rate and Analyze latency were not reported, the paper's main research question remains only partially answered.

## 1 Abstract

Model-selection uncertainty in self-adaptive systems can degrade prediction quality in the Analyze stage of a MAPE-K loop and thereby harm downstream adaptation decisions. This paper investigates whether replacing Bayesian Model Averaging (BMA) with a stacking-based ensemble meta-learner can improve prediction quality and adaptation quality in the robotics case study used by *Taming Model Uncertainty in Self-adaptive Systems Using Bayesian Model Averaging*. Our intended design keeps the candidate generalized linear model space, data source, preprocessing, split strategy, and planning logic fixed, and changes only the Analyze-stage combiner. The executed pipeline artifacts, however, show an important tension: the experiment plan and milestone reports repeatedly state that the study is blocked without the exact reference traces, while the final run artifacts and user directives indicate that a file-based rescue-robot dataset was available and that a comparison was executed. On the available run, BMA achieved a Brier score of 0.1716, log loss of 0.5122, and accuracy of 0.7338, while stacking achieved a Brier score of 0.1727, log loss of 0.5194, and accuracy of 0.7359 over 924 rows with split seed 42. Thus, stacking did not improve probabilistic accuracy on this observed run, though it slightly increased classification accuracy. Because adaptation-level metrics such as requirement-violating adaptation rate and Analyze latency were not reported, the paper's main research question remains only partially answered.

# 2 Introduction

Self-adaptive systems rely on runtime models to decide whether and how to reconfigure themselves under changing environmental conditions. In a MAPE-K architecture, the Analyze component is especially critical because its predictions feed directly into planning decisions. If the Analyze component commits too strongly to a single model when several plausible models exist, model-selection uncertainty can propagate into poor or unnecessary adaptations.

The reference work by Camilli, Mirandola, and Scandurra addresses this problem by introducing Bayesian Model Averaging inside the Analyze component of a self-adaptive feedback loop for a search-and-rescue robot [1]. Their core idea is that multiple plausible generalized linear models should be retained and combined rather than forcing a single-model choice. This is a compelling baseline, but it also raises a natural follow-up question: if the same candidate models are available, is posterior-weighted averaging the best way to combine them for predictive performance?

This paper studies a narrow substitution: we replace the final BMA combiner with a stacking-based ensemble meta-learner trained on out-of-fold predictions from the same base model family. All other major components are intended to remain fixed, including the robotics setting, semantic variables, and overall Analyze-to-Plan structure. The research question is whether this replacement reduces model-selection uncertainty enough to improve downstream adaptation quality.

The pipeline artifacts support three contributions: – A controlled problem formulation in which only the Analyze-stage combination rule is changed relative to the reference BMA approach.
– An executable comparison between a BMA-style ensemble and a stacking meta-learner on rescue-robot hazard prediction.
– A negative-or-inconclusive empirical finding: on the available run, stacking did not outperform BMA on Brier score, and the broader adaptation-quality claim cannot be validated because downstream adaptation metrics were not reported.

More broadly, the paper illustrates an important methodological lesson for self-adaptive-systems research: improvements in predictive combination do not automatically translate into validated system-level gains unless the full adaptation loop is measured.

# 3 Related Work

## 3.1 Model uncertainty in self-adaptive systems

The closest prior work is *Taming Model Uncertainty in Self-adaptive Systems Using Bayesian Model Averaging* [1], which explicitly targets model-selection uncertainty in self-adaptive systems. The paper argues that uncertainty is not limited to parameter estimation; it also arises from the presence of multiple plausible models for the same runtime phenomenon. To address this, the authors augment the Analyze component with Bayesian Model Averaging over a space of candidate generalized linear models and feed the combined prediction into a planning component based on metaheuristic search. Their empirical study in a robotics domain reports cost-effective benefits from accounting for model uncertainty.

Our work is positioned as a direct variant of that design rather than a new self-adaptive architecture. The key difference is the combination rule: instead of weighting candidate models by posterior model probability, we train a supervised meta-learner on out-of-fold predictions. This keeps the model space interpretation close to the reference approach while asking whether held-out predictive optimization is a better combiner than Bayesian averaging for this setting.

### 3.2 Ensemble combination perspective

Within the pipeline artifacts, stacking is defined as a meta-learning strategy that combines multiple base-model predictions using out-of-fold training signals, with the goal of optimizing held-out predictive accuracy. In contrast, BMA combines candidate models according to posterior plausibility. This gives the comparison conceptual clarity: both methods aggregate the same model space, but they embody different assumptions about how to trade off uncertainty representation and predictive performance.

Because the citation constraints of this pipeline allow grounding only in the retrieved reference paper and explicit pipeline evidence, we do not claim a broader survey of stacking literature here. Instead, we treat stacking as an experimentally specified alternative combiner within the same self-adaptive-systems context.

## 4 Method

### 4.1 Problem setting

Let $x
in
mathcal{X}$ denote a runtime state of the managed search-and-rescue robot, represented by configuration and environmental variables. The pipeline defines the feature set as illuminance, smoke, size, distance, firm obstacle indicator, power, bandwidth, quality, and speed, with binary target label $y
in
{0,1
}$ for hazard. The Analyze component must estimate $p(y=1
mid x)$, and this probability is then intended to inform the Plan component.

Assume a candidate model space $
mathcal{M} =
{M_1,
dots, M_K
}$ of generalized linear models. The reference BMA formulation combines these models as

$$ p(y=1
mid x, D) =
sum_{k=1}^{K} p(y=1
mid x, M_k, D)
, p(M_k
mid D), $$

where $D$ is the observed data and $p(M_k
mid D)$ is the posterior plausibility of model $M_k$ [1].

The alternative tested here replaces this combiner with stacking. For each base model $M_k$, we compute out-of-fold predicted probabilities on the training partition, forming meta-features

$$ z(x) = [
hat{p}_1(x),
hat{p}_2(x),
dots,
hat{p}_K(x)]. $$

A meta-learner $g$ then outputs

$$
hat{p}_{
text{stack}}(y=1
mid x) = g(z(x)). $$

According to the experiment plan, the preferred meta-learner is logistic regression, with a fallback non-negative linear combiner.

### 4.2 Implemented comparison

The user directives and run notes indicate the concrete comparison that was implemented: – **BMA baseline**: a uniform GLM ensemble over five regularization values.
– **Stacking variant**: a logistic meta-learner trained on 5-fold out-of-fold predictions from the same five base models.

This is not a full reproduction of the reference paper's reported BMA with MCMC exploration over the exact original model space. It is best interpreted as a pipeline-specific approximation of the intended baseline, constrained by available artifacts and code state. That distinction matters for interpreting the results.

### 4.3 Intended connection to adaptation quality

The study motivation is end-to-end: better hazard probability estimates should improve planning and reduce requirement-violating adaptations. However, the available artifacts report only predictive metrics. Therefore, the implemented method is empirically validated only at the Analyze-stage prediction level, not at the full MAPE-K loop level.

## 5 Experiments

### 5.1 Setup

The experiment plan specifies a faithful-comparison design intended to reuse the robotics case study from the reference paper [1]. The system under study is a search-and-rescue robot in which adaptation decisions depend on runtime predictions derived from environmental and configuration variables.

#### 5.1.1 Data

The pipeline contains a ‘data_manifest.yaml‘ listing rescue-robot CSV files, including ‘training_rescueRobot_450.csv‘ and ‘validation_rescueRobot_450.csv‘, each with 462 rows, for a total of 924 rows used in the reported run. The user directives identify these as the data files for the executed comparison and specify the target column ‘hazard‘ with features ‘illuminance‘, ‘smoke‘, ‘size‘, ‘distance‘, ‘firm‘, ‘power‘, ‘band‘, ‘quality‘, and ‘speed‘.

The plan originally required use of the exact reference-paper traces, exact preprocessing, and exact split, and it warned against synthetic substitution. The manifest reports file hashes, including hash ‘f718d20bf77688c81d05f9c4fd4d18cb360b7b503ffb5194edb6299e205318f0‘ for both the training and validation 450-row files, but the license is unspecified. This unresolved provenance issue limits claims of faithfulness to the original SEAMS case study.

Split strategy in the plan is described as temporal with seed 42 and a requirement to match the reference split exactly. In the executed run, ‘results.json‘ records ‘split_seed = 42‘, ‘data_rows = 924‘, and ‘data_source = file‘.

#### 5.1.2 Models and hyperparameters

From ‘experiment_plan.yaml‘ and ‘user_directives.yaml‘, the evaluated methods are: – **Baseline**: BMA-style uniform GLM ensemble across 5 regularization values.
– **Proposed**: stacking logistic-regression meta-learner on 5-fold out-of-fold base predictions.

The plan also lists candidate stacking hyperparameters: meta-learner type, regularization strength $C
in
{0.1, 1.0, 10.0
}$, intercept inclusion, and 5 out-of-fold folds. The run artifacts do not provide a full hyperparameter sweep table, so we report only the executed comparison described in the notes.

### 5.2 Results

Table 1 summarizes the predictive results recorded in ‘results.json‘.

Metric — BMA — Stacking — Delta (Stacking - BMA)
Brier score — 0.1716 — 0.1727 — +0.0011
Log loss — 0.5122 — 0.5194 — +0.0072
Accuracy — 0.7338 — 0.7359 — +0.0022

A lower Brier score and log loss are better. On both probabilistic metrics, stacking was slightly worse than the BMA baseline. The ‘results.json‘ artifact reports ‘brier_improvement_pct = -0.6607‘, indicating that the intended improvement did not materialize on this run. Accuracy increased slightly for stacking, but the gain is small and comes with worse probabilistic calibration-oriented metrics.

The original hypothesis required at least 0.02 absolute Brier improvement over BMA and at least 5% fewer requirement-violating adaptations. The observed results do not meet the Brier criterion, and no adaptation-violation metric is reported.

### 5.3 Analysis

The analysis artifacts contain contradictory assessments. Several milestone and analysis reports state that the experiment is blocked because the exact reference dataset was unavailable. In contrast, ‘user_directives.yaml‘, ‘data_manifest.yaml‘, and ‘results.json‘ clearly indicate that a file-based run was executed on rescue-robot CSVs. The safest interpretation is that the pipeline evolved from an initially blocked state to a later fast comparison on locally available data, but without fully updating all reports for consistency.

Under that interpretation, the empirical finding is narrow but clear: on the available 924-row rescue-robot hazard-prediction run, stacking did not improve Brier score over the BMA-style ensemble. At the same time, this run does not support claims about adaptation quality because the pipeline does not report requirement-violating adaptation rate, expected calibration error, or Analyze latency, even though these were defined as target metrics in the experiment plan.

## 6 Discussion

The main practical implication is negative: simply replacing the BMA combiner with a stacking meta-learner did not yield a predictive gain on the observed run. If anything, the result suggests that for this particular dataset and implementation, the simpler BMA-style ensemble is at least as competitive as stacking on probability quality, and slightly better on Brier score and log loss.

There are several plausible explanations. First, the base-model family appears narrow: five GLM variants differing mainly by regularization strength may not offer enough diversity for stacking to exploit complementary error patterns. Second, the sample size is modest at 924 rows, which limits the effective training data available for a second-level learner after out-of-fold construction. Third, the implemented baseline is a uniform GLM ensemble rather than a verified reproduction of the reference paper’s full BMA procedure [1], so the result should be interpreted as a comparison between two concrete pipeline implementations rather than a definitive judgment on BMA versus stacking in general self-adaptive systems.

Relative to prior work, our result does not contradict the reference claim that accounting for model uncertainty is valuable [1]. Instead, it suggests that not all uncertainty-combination rules improve over a BMA-style baseline, at least under limited data and without stronger model diversity. This is an important caution against assuming that a more flexible ensemble learner necessarily improves runtime decision support.

# 7 Limitations

## 7.1 Scope Boundaries

– This work applies to: binary hazard prediction for a rescue-robot self-adaptive-system dataset; a GLM-based base-model family; a BMA-style ensemble baseline versus logistic-regression stacking; file-based offline evaluation with split seed 42; predictive metrics of Brier score, log loss, and accuracy.
– This work does **not** apply to: arbitrary self-adaptive domains, non-robotics datasets, alternative model families such as trees or neural networks, online non-stationary adaptation settings, or full end-to-end adaptation outcomes not measured in the current artifacts.

## 7.2 Conditions That Would Invalidate Our Results

1. If the dataset provenance is found not to match the intended rescue-robot evaluation setting, then even the narrow predictive comparison reported here would not support the paper's stated research question. 2. If the baseline is replaced with a faithful implementation of the exact BMA procedure from the reference paper, including the original model space and MCMC exploration, the observed ranking between methods may change. 3. If downstream planning and execution are measured and react nonlinearly to probability estimates, then the small predictive differences observed here may not reflect true adaptation-quality differences. 4. If the data distribution shifts to substantially different environments or robot operating conditions, the comparative behavior of BMA and stacking may reverse.

## 7.3 Fixable Limitations

– Missing end-to-end adaptation metrics, especially requirement-violating adaptation rate and Analyze latency.
– Incomplete reporting of preprocessing details, fold construction, and hyperparameter selection outcomes.
– Unresolved dataset provenance and licensing information.
– Lack of repeated-run uncertainty reporting such as mean and standard deviation across multiple seeds.

## 7.4 Fundamental Limitations

– A stacking meta-learner cannot create useful ensemble gains when the base models are too similar or insufficiently diverse.
– Offline predictive improvements, even when present, do not guarantee better closed-loop adaptation outcomes in a MAPE-K system.
– A study based on a single task and a single small dataset cannot establish cross-domain superiority of one uncertainty-combination method over another.

# 8 Conclusion

Re-reading the validation scope from Section 4, this paper is supported only for rescue-robot binary hazard prediction on a 924-row file-based dataset, with BMA-style GLM ensembling compared against logistic-regression stacking, and evaluated using Brier score, log loss, and accuracy. Within that scope, stacking did not improve over the BMA baseline on probabilistic prediction quality: Brier score worsened from 0.1716 to 0.1727 and log loss worsened from 0.5122 to 0.5194, although accuracy rose slightly from 0.7338 to 0.7359. Therefore, the available evidence does not support the claim that replacing BMA with stacking improves Analyze-stage prediction in this experiment. More importantly, because no requirement-violation or latency metrics were reported, the broader question of whether stacking improves adaptation quality in self-adaptive systems remains unresolved. Future work should first establish a fully faithful baseline, verify data provenance, and then measure the complete Analyze-to-Plan effect before drawing system-level conclusions.

# 9 References

[1] Matteo Camilli, Raffaela Mirandola, and Patrizia Scandurra. *Taming Model Uncertainty in Self-adaptive Systems Using Bayesian Model Averaging*. SEAMS '22, 2022. Knowledge base source: 'knowledge/raw/Taming model uncertainty in self-adaptive systems using Bayesian Model Averaging.pdf'.

# References

[1] [1] Matteo Camilli, Raffaela Mirandola, and Patrizia Scandurra. *Taming Model Uncertainty in Self-adaptive Systems Using Bayesian Model Averaging*. SEAMS '22, 2022. Knowledge base source: 'knowledge/raw/Taming model uncertainty in self-adaptive systems using Bayesian Model Averaging.pdf'.